

A Practical Guide to Adagrad optimizer

TOPIC -- ADAGRAD OPTIMIZER

-- 01 --

$Q(w) = \frac{1}{n} \sum_{i=1}^n Q_i(w) = \frac{1}{n} \sum_{i=1}^n (m(w; x_i) - y_i)^2$, where $m(w; x_i)$ is the predictive model (e.g., a deep neural network) the objective's structure can be exploited to estimate 2nd order information using gradients only. The resulting methods are simple and often effective

-- 02 --

Second-order methods A stochastic analogue of the standard (deterministic) Newton–Raphson algorithm (a "second-order" method) provides an asymptotically optimal or near-optimal form of iterative optimization in the setting of stochastic approximation. A method that uses direct measurements of the Hessian matrices of the summands in the empirical risk function was developed by Byrd, Hansen, Nocedal, and Singer. However, directly determining the required Hessian matrices for optimization may not be possible in practice. Practical and theoretically sound methods for second-order versions of SGD that do not require direct Hessian information are given by Spall and others. (A less efficient method based on finite differences, instead of simultaneous perturbations, is given by Ruppert.) Another approach to the approximation Hessian matrix is replacing it with the Fisher information matrix, which transforms usual gradient to natural. These methods not requiring direct Hessian information are based on either values of the summands in the above empirical risk function or values of the gradients of the summands (i.e., the SGD inputs). In particular, second-order optimality is asymptotically achievable without direct calculation of the Hessian matrices of the summands in the empirical risk function. When the objective is a nonlinear least-squares loss

-- 03 --

This procedure will remain numerically stable virtually for all η as the learning rate is now normalized. Such comparison between classical and implicit stochastic gradient descent in the least squares problem is very similar to the comparison between least mean squares (LMS) and normalized least mean squares filter (NLMS). Even though a closed-form solution for ISGD is only possible in least squares, the procedure can be efficiently implemented in a wide

range of models. Specifically, suppose that $Q_i(w)$ depends on w only through a linear combination with features x_i , so that we can write $\nabla_w Q_i(w) = -q(x_i'w)x_i$, where $q(\cdot) \in \mathbb{R}$ may depend on x_i, y_i as well but not on w except through $x_i'w$. Least squares obeys this rule, and so does logistic regression, and most generalized linear models. For instance, in least squares, $q(x_i'w) = y_i - x_i'w$, and in logistic regression $q(x_i'w) = y_i - S(x_i'w)$, where $S(u) = e^u / (1 + e^u)$ is the logistic function. In Poisson regression, $q(x_i'w) = y_i - e^{x_i'w}$, and so on. In such settings, ISGD is simply implemented as follows. Let $f(\xi) = \eta q(x_i'w_{\text{old}} + \xi x_i)$, where ξ is scalar. Then, ISGD is equivalent to:

-- 04 --

$$w^{\text{new}} := \arg \min_w \{ Q_i(w) + \frac{1}{2} \eta \|w - w^{\text{old}}\|^2 \}.$$

-- 05 --

RMSProp RMSProp (for Root Mean Square Propagation) is a method invented in 2012 by James Martens and Ilya Sutskever, at the time both PhD students in Geoffrey Hinton's group, in which the learning rate is, like in Adagrad, adapted for each of the parameters. The idea is to divide the learning rate for a weight by a running average of the magnitudes of recent gradients for that weight. Unusually, it was not published in an article but merely described in a Coursera lecture.

-- 06 --

Averaging Averaged stochastic gradient descent, invented independently by Ruppert and Polyak in the late 1980s, is ordinary stochastic gradient descent that records an average of its parameter vector over time. That is, the update is the same as for ordinary stochastic gradient descent, but the algorithm also keeps track of

-- 07 --

Approximations in continuous time For small learning rate η stochastic gradient descent $(w_n)_{n \in \mathbb{N}_0}$

x_0 can be viewed as a discretization of the gradient flow ODE

-- 08 --

Backtracking line search Backtracking line search is another variant of gradient descent. All of the below are sourced from the mentioned link. It is based on a condition known as the Armijo–Goldstein condition. Both methods allow learning rates to change at each iteration; however, the manner of the change is different. Backtracking line search uses function evaluations to check Armijo's condition, and in principle the loop in the algorithm for determining the learning rates can be long and unknown in advance. Adaptive SGD does not need a loop in determining learning rates. On the other hand, adaptive SGD does not guarantee the "descent property" – which Backtracking line search enjoys – which is that $f(x_{n+1}) \leq f(x_n)$ for all n . If the gradient of the cost function is globally Lipschitz continuous, with Lipschitz constant L , and learning rate is chosen of the order $1/L$, then the standard version of SGD is a special case of backtracking line search.

-- 09 --

Adam Adam (short for Adaptive Moment Estimation) is a 2014 update to the RMSProp optimizer combining it with the main feature of the Momentum method. In this optimization algorithm, running averages with exponential forgetting of both the gradients and the second moments of the gradients are used. Given parameters $w(t)$ and a loss function $L(t)$, where t indexes the current training iteration (indexed at 1), Adam's parameter update is given by: